

Lab 1: MIPS R10000 and Exploiting ILP

This exercise is based on and refers to the *MIPS R10000 Superscalar Microprocessor* document, which can be found on the course web page.

1 Pipeline and Register Renaming

Question 1.1. Explain why in the 3rd stage of the pipeline (see Figure 2) the *register file* is read in the 2nd half of each cycle. What are the advantages and disadvantages of such an implementation?

Answer. The write back is made in the first half of a cycle and the read is done in the 2nd half, allowing to use data written back during the same cycle, thus avoiding forwarding paths. The main drawback is that read and write should be done in half a clock cycle. Therefore the implementation of the register file is critical because it must work twice as fast as the other components.

Question 1.2. Figure 5 describes register renaming. Why are the destination and source registers represented with 5 bits in the original instruction, and with 6 bits after the renaming?

Answer. In the original instruction, the programmer is addressing the logical registers. There are 32 logical floating-point registers in the R10000, thus 5 bits are needed to address them. After register renaming, the processor is addressing physical registers, there are 64 physical floating-point registers in the processor (6 bits). These registers are used to allow parallel out of order execution of several independent instructions that are using the same logical destination registers.

Question 1.3. Specify the function of the *queues*, *active lists*, and *map tables* in the R10000. Relate each of these to the structures described in the course (*reservation stations*, *ROB*, ...).

Answer. The Queues are used to store decoded instructions before they are processed in the execution units. The active lists stores information about all the instructions that are decoded and not yet graduated. The map tables are used to know how the physical registers are assigned to the logical ones.
 , Queues are Reservation Station and Active Lists are used as Reorder Buffers.

- Address Queue = Load Store Queue
- Active list = Reorder Buffer
- Int/FP Queue = Reservation Station
- Register Map Table = Mapping Table

Question 1.4. Describe the role of the components *Rdy*, *OpA*, *OpB*, *OpC*, *Dest*, *Old Dest*, *Tag*, and *D* in Figure 5.

Answer. See below:

- *Rdy* is 3 bits wide, each bit refers to a source register of the instruction. It gives the state of these registers, to know whether the value inside them is correct or not. The instruction can be processed only when the 3 bits are set, which means that all the operands are available.
- *OpA*, *OpB* and *OpC* refer to the physical registers that will be used as the source of the operation.
- *Dest* is the physical register where the result will be written.
- *Old Dest* is the previous physical register mapped to the same logical destination register as *Dest*. It is kept so that, (1) on commit, the old physical register can be freed, and (2) on an exception, the previous logical-to-physical mapping can be restored.
- *Tag* gives the address where the instruction is stored in the active list. It is used to match lines in the queue and in the active list together.
- *D* is the done bit; it says when the instruction has achieved its operation.

Question 1.5. How many memory bits are needed for the *FP queue*, the *active list* (without the CC bits), the *FP register map table*, the *free register list*, and the *FP busy bit table*? Justify your answers.

Answer.

In total, we need 1824 bits:

- Tag = 5 bits, Rdy = 3 bits, OpA = OpB = OpC = 6 bits, Dest = 6 bits, Func = 10, Unit = 4, Br = 4. 50 bits per line. 16 entries in the FP queue, so 800 bits are needed in the FP Queue.
- OldDest = 6 bits, LogDest = 5 bits and Done = 1 bit, Exc = 6, so 18 bits per line, there are 32 entries, so 576 bits are needed for the Active List.
- 32 entries of 6 bits, 24 bytes are needed for the FP register Map Table.
- 32 entries of 6 bits, so 24 bytes are needed for the Free Register List.
- There are 64 registers, so the processor needs 64 busy bits.

Question 1.6. How many read ports are there in the *FP busy bit table*? How many read ports and how many write ports are there in the *FP map table*? Justify your answers.

Answer. Each instruction needs to read the busy status of 3 registers. 4 instructions are decoded each cycle. 3 extra ports are added for special instructions. So there are 15 read ports.

Since each cycle, up to 4 instructions are decoded and each instruction have 3 operand register and 1 destination register, the FP map tables need 16 read ports and 4 write ports.

The write port is used to update the mapping of the logical destination register.

Question 1.7. Explain how each kind of dependency is detected and how each is handled.

(become writes to different registers)

Answer. WAW and WAR dependencies disappear because of register renaming. RAW dependencies are detected by reading the busy bit table, the busy bit of each physical register indicates if there is a dependency on this register.

Question 1.8. Using the tables in annex, describe the behaviour of the processor executing the following program¹. You should complete the *active lists*, the *FP queue* and *address queue*, the *FP register map table*, the *busy bit table*, the *free list*, and the position of the instructions in the pipeline for each cycle. Use one copy of the tables per cycle (in Excel, use a separate sheet for each cycle). Write all numbers in hexadecimal. Use the following hints and assumptions to simplify the simulation:

- Assume the data and instruction caches never raise a miss.
- Assume all queues are empty at the beginning.
- Assume integer register 1 (\$I1) is mapped to physical register (\$I0x01).
- Assume all instructions after #00000010 are NOPs.
- Assume that after issue, all instructions spend exactly one cycle in each applicable stage—even, e.g., the SQRT.S instruction.
- Assume floating point results are first available in the write-back stage.
- Remember that internal components are usually structured as they are for a good reason (or reasons).
- Remember that creating new register mappings, removing newly-allocated registers from the free list, and marking those registers as busy must all occur in the same cycle.
- Remember to carefully consider the timing and sequence of register write-back, instruction issue, removal from instruction queues, and active list updates.

#00000001	LDC1	\$F0,	#0000(\$I1)
#00000002	LDC1	\$F1,	#0001(\$I1)
#00000003	LDC1	\$F2,	#0002(\$I1)
#00000004	LDC1	\$F3,	#0003(\$I1)
#00000005	MUL.S	\$F4,	\$F0, \$F3
#00000006	MUL.S	\$F5,	\$F1, \$F2
#00000007	SUB.S	\$F4,	\$F4, \$F5
#00000008	ADD.S	\$F5,	\$F0, \$F1
#00000009	MUL.S	\$F9,	\$F5, \$F5
#0000000A	MUL.S	\$F10,	\$F4, \$F0
#0000000B	SUB.S	\$F9,	\$F9, \$F10
#0000000C	SQRT.S	\$F9,	\$F9
#0000000D	SDC1	\$F4,	#0004(\$I1)
#0000000E	SDC1	\$F5,	#0005(\$I1)
#0000000F	SDC1	\$F9,	#0006(\$I1)
#00000010	SDC1	\$F10,	#0007(\$I1)

Answer. Correction in annex.

LDC1 = load double word from memory
¹ SDC1 = store double word to memory
 instr.S = floating point instruction (example multiplication = MUL.S)

2 Exception Handling

Question 2.1. When an exception occurs, the processor must restore its state to exactly match the programmer's expectation precisely when the programmer expects that exception to arise (for example, all the instructions executed after the faulty instruction should be discarded). Explain how the R10000 achieves this goal.

- Discards all instructions after the one triggering exception in the ROB (program order)
- Unmaps 4 instructions per cycle: Need to check the oldDest of the Map table each time to restore mappings
- Waits for instructions before the one triggering exception to be done before jumping to exception handler

Answer. When an exception occurs, the processor discards all instructions that are after the exception by unmapping them in reverse order. In that way it can also restore its state. It unmaps 4 instructions per cycle. The exception handling is quite slow compared to branches but it is much rarer and does not affect overall performance too much.

Question 2.2. In the previous program, what happens if instruction #00000006 generates an overflow exception? Describe how the processor restores its state according to information it has in the queues and active lists and why the state is restored in this way.

Answer. An exception is raised; the processor unmaps all instructions after the DIV to restore its state at the exception. It uses the information of the previous registers located in the active lists to restore correct register mapping. It unmaps 4 instruction per cycle in reverse order to keep track of multiple register renamings of a single logical register.

3 Branch Prediction

Question 3.1. The R10000 is able to predict branches. What kind of algorithm does it use? Is it dynamic or static prediction? Describe the state machine the algorithm uses.

Answer. The processor makes dynamic prediction using a 2-bit algorithm based on a 512-entry branch history table. This table is indexed with bits 11:3 of the address of the branch instruction, since we need 8 bits to address the table and the address are aligned every 32 bits (the first 2 bits are 0). Figure 1 shows the state diagram of the algorithm.

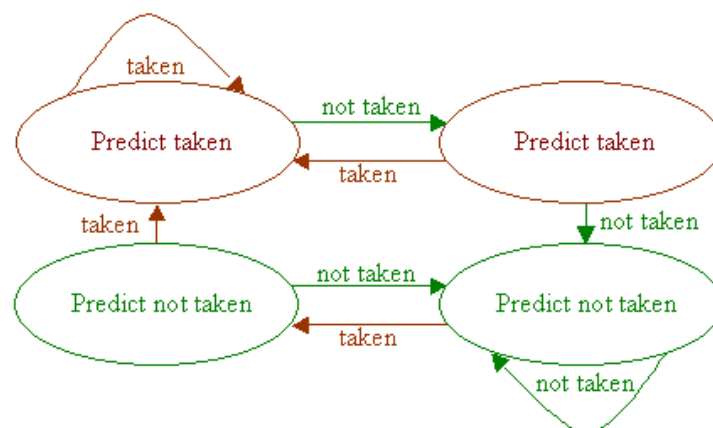


Figure 1: 2-bit algorithm state diagram

Question 3.2. What happens if the prediction is not correct? Describe the processor's behaviour in detail.

Answer. If the prediction is not correct, the processor has to restore its state as at the time of the branch, discard all mispredicted instructions and resume execution along the correct path. The processor can predict 4 branches. Each time a prediction is done it stores its state in the branch stack, which contains the alternative branch address (the correct one in case of a misprediction), the complete copies of the integer

and floating-point map tables (to restore correct register values) and other control bits. Every instruction executed along a predicted path cannot graduate before the branch direction is known. When a misprediction occurs, the processor restores the map tables, discards all instruction of the wrong path and jumps to the correct address. A misprediction introduces a branch bubble in the processor: in the cycle following the misprediction detection no instructions are decoded.

Question 3.3. Explain why there is a NOP after the branch in the following program. Change the program to avoid such an instruction.

```

                                #00000001      LD      $I1,  #0000($I10)
                                #00000002      SUB      $I17, $I17, $I17
loop:                          #00000003      LD      $I5,  #0000($I2)
                                #00000004      LD      $I6,  #0000($I3)
                                #00000005      MUL      $I5,  $I6, $I5
                                #00000006      SD      $I5,  #0000($I4)
                                #00000007      SUBI     $I1,  $I1, 1
                                #00000008      ADDI     $I2,  $I2, 4
                                #00000009      ADDI     $I3,  $I3, 4
                                #0000000A      ADDI     $I4,  $I4, 4
                                #0000000B      BEQ      $I1,  $I17, loop
                                #0000000C      NOP
                                #0000000D      LD      $I1,  #0000($I10)
                                #0000000E      MULI     $I1,  $I1, 4
                                #0000000F      SUB      $I4,  $I4, $I1

```

Answer. In the MIPS ISA, the instruction immediately following a branch or a jump is always executed (a delayed branch). In this case the processor executes nothing (NOP). We could remove the NOP and put the instruction just before the branch (ADDI \$I4, \$I4, 4) in its place, since the branch does not depend on that instruction.